

МАШИННОЕ ОБУЧЕНИЕ

для начинающих

Учебник для тех, кто хочет научить компьютер думать

Практический курс · от первых данных до своего проекта
Python · Pandas · Scikit-learn · нейросети · LLM

Содержание

Как читать эту книгу

Эта книга — не сборник формул и не толстый справочник, который страшно открывать. Это **маршрут**: от «я вообще не понимаю, что такое машинное обучение» до момента, когда ты запускаешь собственное приложение и отправляешь ссылку на него друзьям.

Ты будешь не читать про машинное обучение, а заниматься им. Почти в каждой главе есть код, который можно запустить прямо в браузере, и задание, которое стоит сделать самому.

Правило простое: если можно попробовать — попробуй.

Что нужно знать заранее

Мы предполагаем, что ты уже писал простые программы на Python: знаешь, что такое переменная, список, цикл и функция. Если ты когда-нибудь делал запрос к базе данных (SQL) — вообще отлично, это пригодится. Всё остальное объясняется с нуля.

Как устроена книга

Книга разбита на шесть частей — мы называем их фазами. Каждая следующая фаза опирается на предыдущую, поэтому читай по порядку.

Шесть фаз пути

Фаза 1 — Данные и ML-мышление. Учимся смотреть на данные и находить в них закономерности.

Фаза 2 — Классический ML. Строим первые модели, которые предсказывают.

Фаза 3 — Нейросети. Разбираемся, как машина «видит» и учится на примерах.

Фаза 4 — Большие языковые модели (LLM). Понимаем, как работает то, что стоит за ChatGPT.

Фаза 5 — AutoML и веб-приложения. Превращаем модель в продукт с кнопкой.

Фаза 6 — Финальный проект. Делаем и защищаем что-то своё.

Значки, которые встретятся в тексте

Аналогия

Так помечены сравнения из обычной жизни — они помогают понять сложную идею быстро.

Внимание, частая ошибка

Так отмечены места, где почти все спотыкаются. Прочитай внимательно — сэкономишь часы.

Полезно знать

Дополнение или уточнение, которое углубляет понимание, но не обязательно для первого прохода.

Где запускать код

Весь код в книге рассчитан на **Google Colab** — бесплатную среду, которая работает прямо в браузере. Ничего устанавливать не нужно: заходишь на colab.research.google.com, создаёшь новый блокнот и вставляешь код. Всё считается на серверах Google.

Готов? Тогда начнём с главного вопроса: чем машинное обучение вообще отличается от обычного программирования.

ФАЗА 1

Данные и ML-мышление

Учимся видеть закономерности в данных — фундамент всего, что будет дальше.

Глава 1. «Магия» машинного обучения

Две разные идеи: правила и обучение

Представь, что тебя попросили написать программу, которая отличает кошку от собаки на фотографии. Как бы ты это сделал обычным способом — через правила?

Ты бы начал: «Если уши треугольные и торчат вверх — это кошка. Если морда вытянутая — собака...» И очень быстро увяз бы. А вислоухая кошка? А чихуахуа с торчащими ушами? А фото сбоку? Правил становятся тысячи, и они всё равно не работают.

Машинное обучение переворачивает задачу. Вместо того чтобы писать правила, ты показываешь машине **тысячи примеров** — «вот кошка», «вот собака», «вот снова кошка» — и она сама находит закономерности, которые отличают одно от другого. Правила она выводит без тебя.

Аналогия: рецепт и дегустация

Обычное программирование — это рецепт: ты записываешь точные шаги, и повар их выполняет. Машинное обучение — это дегустатор, который попробовал сотни блюд и теперь на вкус угадывает, солёное оно или сладкое, хотя рецепта ему никто не давал.

Определение своими словами

Главная мысль главы

Машинное обучение — это способ научить компьютер решать задачу не через записанные правила, а через примеры, из которых он сам выводит закономерности (паттерны).

Что такое «паттерн в данных»

Паттерн — это устойчивая закономерность. Люди находят паттерны постоянно, даже не замечая. Ты знаешь, что после серых туч часто идёт дождь. Никто не давал тебе формулу «туча → дождь» — ты вывел её из наблюдений. Именно этим занимается модель машинного обучения, только с числами.

Например, в данных о жилье может прятаться паттерн: «чем больше площадь, тем выше цена». В данных о фильмах: «фильмы с рейтингом выше 8 чаще собирают большую кассу». Модель ищет такие связи и потом использует их, чтобы делать **предсказания** на новых данных.

Типы задач машинного обучения

Почти всё, что мы будем делать, сводится к трём большим типам задач. Разберём каждый на понятном примере.

1. Классификация — «к какой категории?»

Модель выбирает одну из нескольких категорий. Спам или не спам. Кошка, собака или птица. Положительный отзыв или отрицательный. Ответ — это метка (label).

2. Регрессия — «сколько?»

Модель предсказывает число. Какой будет цена квартиры. Сколько баллов наберёт ученик. Какая температура будет завтра. Ответ — это число на шкале, а не категория.

3. Кластеризация — «кто на кого похож?»

Модель сама, без подсказок, разбивает данные на группы похожих объектов. Например, находит среди покупателей группы со схожим поведением. Здесь никто заранее не говорит модели правильный ответ — она группирует сама.

Тип задачи	Вопрос	Пример ответа
Классификация	К какой категории?	спам / не спам
Регрессия	Сколько? Какое число?	цена = 4 200 000
Кластеризация	Кто на кого похож?	группа А, В, С

Внимание, частая ошибка

Классификацию и регрессию путают чаще всего. Запомни разницу по вопросу: если ответ — это категория (одна из нескольких), это классификация. Если ответ — число на непрерывной шкале, это регрессия. «Сдаст экзамен / не сдаст» — классификация. «Сколько баллов наберёт» — регрессия.

Обучение с учителем и без учителя

Классификация и регрессия — это обучение с учителем (supervised): в данных есть правильные ответы, на которых модель тренируется. Кластеризация — обучение без учителя (unsupervised): правильных ответов нет, модель ищет структуру сама. В этом курсе мы почти всё время работаем с обучением с учителем — оно нагляднее и понятнее в начале пути.

Попробуй сам: игра «Угадай правило»

Эту игру можно сыграть без единой строчки кода — с другом или всем классом. Она показывает суть машинного обучения буквально за пять минут.

1. Один игрок загадывает секретное правило, например: «число проходит, если оно чётное».
2. Второй игрок называет числа: 4 → проходит, 7 → не проходит, 10 → проходит...
3. С каждым ответом второй игрок уточняет свою гипотезу о правиле — ровно как модель уточняет закономерность на новых примерах.
4. Когда второй уверен — он называет правило. Угадал? Значит «обучение» удалось.

Ты только что сработал как модель машинного обучения: получал примеры с ответами и постепенно выводил скрытое правило. Именно это делает алгоритм — только с тысячами примеров и очень быстро.

Проверь себя

1. Чем машинное обучение отличается от программирования правилами? Объясни в одном предложении.
2. Приведи по одному своему примеру задачи классификации и задачи регрессии.
3. Задача «разбить учеников школы на группы по интересам, не зная заранее какие» — это какой тип обучения?

Глава 2. Данные вокруг нас: знакомство с Pandas

Машинное обучение начинается не с моделей, а с данных. Прежде чем что-то предсказывать, нужно научиться данные открывать, читать и понимать. Главный инструмент для этого в Python называется Pandas.

Аналогия: Pandas — это Excel внутри Python 💡

Если ты когда-нибудь работал с таблицей в Excel или Google Таблицах — ты уже почти понимаешь Pandas. Это тот же самый лист с колонками и строками, только вместо мышки ты управляешь им кодом. Кода бояться не нужно: команд немного, и они читаются почти как английские фразы.

Три главных слова: датасет, признак, целевая переменная

Датасет — это таблица с данными. Каждая **строка** — один объект (например, один пассажир корабля). Каждый **столбец** — одна характеристика объекта, её называют **признаком** (feature): возраст, пол, цена билета.

Среди столбцов часто есть один особенный — тот, который мы хотим предсказывать. Его называют **целевой переменной** (target). Например, «выжил пассажир или нет». Все остальные признаки — это подсказки, по которым модель будет угадывать целевую переменную.

	Возраст (признак)	Пол (признак)	Выжил (цель)
Пассажир 1	22	муж.	нет
Пассажир 2	38	жен.	да
Пассажир 3	26	жен.	да

Первый код: открываем датасет

Загрузим известный учебный датасет о пассажирах «Титаника» — мы будем возвращаться к нему на протяжении нескольких глав. Вставь этот код в новую ячейку Google Colab и запусти.

```
# Импортируем библиотеку Pandas под коротким именем pd
import pandas as pd

# Загружаем датасет "Титаник" прямо из интернета по ссылке
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url) # df — сокращение от DataFrame ("таблица")

# Показываем первые 5 строк таблицы
df.head()
```

Ожидаемый результат: таблица из 5 строк с колонками Name, Sex, Age, Survived и другими.

Три команды, чтобы понять любой датасет

Как только данные загружены, три команды дают полную картину. Запускай каждую в отдельной ячейке.

df.head() — как выглядят данные

Показывает первые несколько строк. Это первое, что делают всегда: буквально посмотреть глазами, что за таблица перед тобой, какие есть колонки и что в них лежит.

df.info() — что за колонки и нет ли пропусков

```
df.info()
```

Показывает список колонок, их тип (число или текст) и сколько значений заполнено. Если в колонке Age заполнено 714 значений из 891 — значит, 177 пропущено. Это важный сигнал, к которому мы вернёмся в главе 4.

df.describe() — статистика по числам

```
df.describe()
```

Для каждой числовой колонки показывает среднее, минимум, максимум и другие величины. Один взгляд — и ты знаешь, что средний возраст пассажира около 30 лет, а самому младшему меньше года.

Фильтрация и группировка: привет из SQL

Если ты знаком с SQL, следующие операции покажутся родными. Pandas умеет то же самое, что запросы к базе данных, только внутри Python.

Фильтрация — как WHERE

```
# Оставить только пассажиров женского пола
women = df[df["Sex"] == "female"]
women.head()
```

Это прямой аналог SQL-запроса `SELECT * FROM df WHERE Sex = 'female'`. Внутри квадратных скобок мы пишем условие, и остаются только строки, для которых оно истинно.

Группировка — как GROUP BY

```
# Средняя доля выживших отдельно для мужчин и женщин
df.groupby("Sex")["Survived"].mean()
```

Эта одна строка отвечает на настоящий вопрос: кто выживал чаще? Результат обычно поражает — среди женщин доля выживших намного выше. Это и есть паттерн в данных, найденный за секунду.

Внимание, частая ошибка ⚠

Легко перепутать одиночное = (присваивание) и двойное == (сравнение). В условии фильтра всегда двойное: `df["Sex"] == "female"`. Одиночное = здесь вызовет ошибку.

Домашнее задание

Зайди на сайт Kaggle (kaggle.com/datasets) и найди датасет на тему, которая тебе интересна: игры, музыка, спорт, кино. Загрузи его в Colab через `pd.read_csv` и ответь на три вопроса:

- Сколько в нём строк и столбцов? (подсказка: `df.shape`)
- Какие есть признаки и есть ли среди них пропуски? (`df.info()`)
- Найди один интересный факт с помощью `groupby` — например, среднее значение чего-то по категориям.

Глава 3. Визуализация и разведка данных (EDA)

Таблица из тысяч чисел ничего не говорит человеку напрямую. А один хороший график — говорит сразу всё. Процесс, когда мы разглядываем данные через графики и ищем в них закономерности, называется **разведочным анализом данных** (EDA, Exploratory Data Analysis). Это как осмотреться на новом месте, прежде чем действовать.

Три графика, которые нужны почти всегда

Гистограмма — как распределены значения

Гистограмма показывает, каких значений в данных много, а каких мало. Например, каких возрастов среди пассажиров больше всего.

```
import matplotlib.pyplot as plt # библиотека для графиков

# Гистограмма возрастов пассажиров
df["Age"].hist(bins=20)         # bins — на сколько столбиков делить
plt.xlabel("Возраст")
plt.ylabel("Сколько пассажиров")
plt.title("Распределение возрастов")
plt.show()
```

Ожидаемый результат: столбчатая диаграмма — видно, что больше всего пассажиров в возрасте 20–35 лет.

Диаграмма рассеяния (scatter plot) — связь двух признаков

Каждая точка — один объект. По одной оси один признак, по другой — второй. Так видно, связаны ли они. Если точки складываются в линию — связь есть.

```
# Есть ли связь между возрастом и ценой билета?
df.plot.scatter(x="Age", y="Fare")
plt.show()
```

Тепловая карта корреляций (heatmap) — что с чем связано

Показывает сразу для всех пар признаков, насколько сильно они связаны. Понадобится Seaborn — библиотека красивых графиков поверх Matplotlib.

```
import seaborn as sns

# Считаем корреляции только по числовым колонкам
corr = df.corr(numeric_only=True)

# Рисуем тепловую карту: чем ярче клетка, тем сильнее связь
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()
```

Что такое корреляция

Аналогия: качели

Корреляция — это число от -1 до +1, показывающее, как связаны два признака. +1: когда один растёт, другой тоже растёт (как рост и вес). -1: когда один растёт, другой падает (как цена билета и класс каюты, где 1-й класс дороже 3-го). Около 0: связи нет, ведут себя независимо.

Внимание, частая ошибка

Корреляция — это не причина! Если мороженое и солнечные ожоги растут вместе, это не значит, что мороженое вызывает ожоги. Просто оба зависят от жаркой погоды. Всегда спрашивай себя: нет ли третьей причины?

Главное правило EDA: график без вывода бесполезен

Построить график — половина дела. Вторая половина — сформулировать словами, что он показывает. Возьми за привычку: под каждым графиком писать вывод одним предложением. «На гистограмме видно, что большинство пассажиров — молодые люди 20–35 лет».

Мини-конкурс: найди самый необычный инсайт

Возьми свой датасет из прошлого задания и построй на нём три разных графика. Для каждого напиши вывод словами. Затем выбери самый неожиданный факт — то, чего ты не ожидал увидеть. Это и есть инсайт: то, что данные рассказали тебе, а ты не знал заранее.

Проверь себя

1. Какой график выберешь, чтобы понять, как распределён возраст в классе?
2. Корреляция между двумя признаками равна $-0,8$. Что это значит?
3. Почему нельзя говорить «признак А вызывает признак В» только на основании высокой корреляции?

Глава 4. Мини-проект «Расскажи о данных»

Пришло время собрать всё из первой фазы вместе. Это твой первый мини-проект — небольшое, но полное исследование, которое можно показать друзьям и родителям. Здесь мы также научимся справляться с двумя частыми проблемами реальных данных: пропусками и выбросами.

Проблема 1: пропущенные значения

Реальные данные почти всегда неполные. В «Титанике» у многих пассажиров не указан возраст. Модель не умеет работать с пустыми клетками, поэтому пропуски нужно как-то обработать.

```
# Сколько пропусков в каждой колонке?
df.isnull().sum()

# Вариант 1: удалить строки с пропусками (осторожно — теряем данные)
df_clean = df.dropna()

# Вариант 2: заполнить пропуски средним значением (чаще лучше)
df["Age"] = df["Age"].fillna(df["Age"].median())
```

Полезно знать

Заполнять пропуски медианой (серединным значением), а не средним, безопаснее: медиана не «съезжает» из-за нескольких очень больших или очень маленьких значений.

Проблема 2: выбросы

Выброс (outlier) — это значение, которое сильно выбивается из общего ряда. Например, если у одного пассажира указана цена билета в 10 раз выше всех остальных. Выбросы могут быть как ошибкой ввода, так и настоящим редким случаем. Найти их помогает график типа «**ящик с усами**» (boxplot).

```
import matplotlib.pyplot as plt

# Ящик с усами по цене билета: точки далеко от "ящика" — выбросы
df.boxplot(column="Fare")
plt.show()
```

Структура твоего мини-проекта

Оформи исследование как небольшую презентацию из трёх частей. На выступление хватит трёх минут.

1. О данных. Что за датасет, сколько строк и столбцов, какие признаки. Один слайд.
2. Что нашёл. 2–3 графика с выводами словами и главный инсайт. Один-два слайда.
3. Вывод. Один содержательный факт, который данные рассказали тебе. Один слайд.

Как оценивается мини-проект

Мини-проект оценивается по простой рубрике — она помогает понять, что важно, а не для отметки.

Критерий	Баллы	Что оценивается
Постановка	0–2	Понятно, о каких данных речь
Работа с данными	0–3	Обработаны пропуски и выбросы
Визуализация	0–2	Графики уместны и подписаны
Объяснение	0–3	Есть вывод словами и инсайт

Главная мысль фазы 1

Ты научился главному навыку любого специалиста по данным: взять сырую таблицу, привести её в порядок, разглядеть в ней закономерности и рассказать о них человеческим языком. Теперь мы готовы сделать следующий шаг — научить машину предсказывать.

ФАЗА 2

Классический ML

Строим первые модели, которые предсказывают, и учимся честно проверять их качество.

Глава 5. KNN: алгоритм ближайших соседей

В первой фазе мы разглядывали данные. Теперь научим машину предсказывать. Начнём с самого понятного алгоритма во всём машинном обучении — метода k ближайших соседей, коротко KNN (k-Nearest Neighbors).

Аналогия: скажи мне, кто твой сосед

KNN — это «ленивый ученик». Он ничему не учится заранее. Когда нужно классифицировать новый объект, он просто смотрит: а на кого этот объект похож среди тех, кого я уже знаю? Находит несколько самых близких «соседей» и повторяет их ответ. Скажи мне, кто твои соседи, — и я скажу, кто ты.

Как это работает по шагам

Представь точки на плоскости: синие — «сдал экзамен», красные — «не сдал». Появляется новая точка без цвета. Что делает KNN?

1. Измеряет расстояние от новой точки до всех известных точек.
2. Выбирает k ближайших (например, $k = 5$ — пять самых близких).
3. Смотрит, какого цвета большинство из этих пяти соседей.
4. Присваивает новой точке цвет большинства. Голосование окончено.

Параметр K — сколько соседей спрашивать

K — это главная настройка алгоритма. Она задаёт, скольких соседей мы опрашиваем. И выбор K сильно меняет поведение модели.

Значение K	Что происходит	Риск
K = 1	Смотрим только на одного ближайшего соседа	Модель шумит, реагирует на случайные точки
K средний (5–15)	Голосует небольшая группа соседей	Обычно оптимально
K очень большой	Голосует почти весь датасет	Модель «размывается», теряет детали

Полезно знать

K обычно берут нечётным (3, 5, 7...) — чтобы при голосовании не случилось ничьей. Как правильно подбирать K по-честному, разберём в следующей главе.

Живая игра без кода

Расставьте по классу учеников двух «типов» — например, любителей спорта и любителей музыки — в разных углах комнаты. Пусть новый ученик встанет где-то посередине и посмотрит на трёх ближайших к себе. Кого больше рядом — к той группе KNN его и отнесёт. Вы только что стали живым алгоритмом.

Первый классификатор в коде

Соберём KNN на данных о пассажирах «Титаника»: предскажем, выжил пассажир или нет, по его возрасту и цене билета.

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
```

```
# Загружаем данные и убираем строки с пропусками (пока просто так)
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url).dropna(subset=["Age", "Fare", "Survived"])

# X — признаки (по чему предсказываем), y — цель (что предсказываем)
X = df[["Age", "Fare"]]
y = df["Survived"]

# Создаём модель: спрашиваем 5 ближайших соседей
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X, y)          # "обучаем" — KNN просто запоминает точки

# Предсказываем для нового пассажира: возраст 25, билет за 30
print(model.predict([[25, 30]])) # 1 = выжил, 0 = нет
```

Ожидаемый результат: [0] или [1] — прогноз для выдуманного пассажира.

Внимание, частая ошибка ⚠

KNN измеряет расстояние — значит, ему важен масштаб признаков. Возраст (0–80) и цена билета (0–500) в разных диапазонах, и цена «перевесит» возраст просто потому, что числа крупнее. Это лечится масштабированием признаков — к нему мы вернёмся в главе 10, когда соберём Pipeline.

Проверь себя ⓘ

1. Почему KNN называют «ленивым» алгоритмом?
2. Что произойдёт с моделью, если задать $K = 1$?
3. Почему KNN чувствителен к масштабу признаков?

Глава 6. Честная проверка: train/test, baseline и утечка данных

Это, возможно, самая важная глава всего курса. Здесь речь не о новом алгоритме, а о том, как понять — хороша модель или мы себя обманываем. Без этого любой красивый результат ничего не стоит.

Разделение на train и test

Аналогия: зазубрил или понял 💡

Представь ученика, который выучил ответы к конкретным задачам наизусть. На этих задачах он получит 100%. Но дай ему новую задачу — и он провалится. Он зазубрил, а не понял. Модель ведёт себя точно так же: если проверять её на тех же данных, на которых она училась, она покажет отличный результат, но это ничего не говорит о новых данных.

Поэтому данные всегда делят на две части. **Обучающая выборка (train)** — на ней модель учится. **Тестовая выборка (test)** — её модель никогда не видела при обучении, и именно на ней мы честно проверяем качество. Обычно берут 80% на обучение и 20% на тест.

```
from sklearn.model_selection import train_test_split

# test_size=0.2 → 20% данных уходит в тест
# random_state=42 → фиксируем случайность, чтобы результат повторялся
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)          # учимся ТОЛЬКО на train

from sklearn.metrics import accuracy_score
pred = model.predict(X_test)         # проверяем на невиданном test
print("Accuracy:", accuracy_score(y_test, pred))
```

Ожидаемый результат: доля правильных ответов на тесте, например 0.68.

Внимание, частая ошибка ⚠️

Проверять модель на обучающих данных — ошибка номер один у новичков. Результат будет завышенным и бесполезным. Правило простое: тест модель не видит до самого конца. И всегда ставь `random_state` — иначе при каждом запуске разбиение будет разным, и результаты не сравнить.

Baseline: с чем вообще сравнивать?

Допустим, модель дала accuracy 0,68. Это хорошо или плохо? Ответить невозможно, пока нет **точки отсчёта**. Такой точкой служит baseline — самая тупая модель, какую можно придумать. Например: «всегда предсказывай самый частый класс».

Если на «Титанике» погибло 62% пассажиров, то модель, которая тупо всем ставит «погиб», получит accuracy 0,62. Значит, наши 0,68 — это плюс всего 6 пунктов к тупому угадыванию. Baseline мгновенно показывает, стоит ли модель вообще чего-то.

```
from sklearn.dummy import DummyClassifier
```

```
# "most_frequent" — всегда предсказывает самый частый класс
baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
print("Baseline accuracy:", baseline.score(X_test, y_test))
```

Data leakage — коварная утечка данных

Аналогия: подсмотреть ответы 💡

Утечка данных (data leakage) — это когда модель случайно «подсматривает» в тестовые данные или в информацию, которой в реальности не будет. Это как если бы ученик готовился к экзамену по украденным заранее ответам: на экзамене он блеснёт, а знаний нет.

Классический пример утечки: предсказываем, вернёт ли клиент кредит, и случайно оставляем в признаках колонку «сумма штрафа за невозврат». Но штраф появляется только после невозврата! В реальный момент предсказания его ещё нет. Модель покажет фантастическую точность на данных, но в жизни окажется бесполезной.

Полезно знать ⓘ

Ещё один частый источник утечки — масштабировать или заполнять пропуски на всех данных сразу, до разделения на train/test. Тогда информация из теста «протекает» в обучение. Именно поэтому в главе 10 мы соберём Pipeline: он делает все преобразования правильно и защищает от этой ошибки автоматически.

Соревнование: честный подбор K

Теперь мы умеем проверять честно. Попробуй разные значения K и найди лучшее — но оценивай только по тесту, а не по обучению.

```
for k in [1, 3, 5, 7, 15, 30]:
    m = KNeighborsClassifier(n_neighbors=k)
    m.fit(X_train, y_train)
    acc = m.score(X_test, y_test)
    print(f"K={k}: accuracy={acc:.3f}")
```

Обрати внимание: K=1 обычно даёт идеальный результат на train, но слабый на test — это первый звоночек переобучения, о котором подробно в следующей главе.

Проверь себя ⓘ

1. Зачем делить данные на train и test?
2. Модель даёт accuracy 0,55, а baseline — 0,60. Что это говорит о модели?
3. Приведи свой пример утечки данных.

Глава 7. Деревья решений: компьютер-детектив

KNN сравнивал объекты по схожести. Дерево решений действует иначе — оно задаёт данным вопросы, шаг за шагом сужая круг, как детектив на допросе.

Аналогия: игра «20 вопросов» 💡

Ты загадал персонажа, а друг угадывает через вопросы «да/нет»: «Это человек? Он реальный? Он спортсмен?» Каждый вопрос отсекает половину вариантов. Дерево решений делает ровно это: задаёт признакам вопросы вроде «Пол = женский?», «Возраст < 12?» — и с каждым ответом приближается к предсказанию.

Узлы, ветви и листья

Дерево состоит из **узлов** (в каждом — вопрос по одному признаку), **ветвей** (ответы «да»/«нет») и **листьев** (финальные предсказания). Алгоритм сам выбирает, какой вопрос задать первым: тот, что лучше всего разделяет данные на группы. Это и есть «информативность вопроса».

Дерево в коде и его картинка

Главное достоинство деревьев — их можно нарисовать и понять глазами. Построим дерево на «Титанике».

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# max_depth=3 — ограничиваем глубину, чтобы дерево было читаемым
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)

# Рисуем дерево
plt.figure(figsize=(14, 6))
plot_tree(tree, feature_names=["Age", "Fare"],
          class_names=["погиб", "выжил"], filled=True)
plt.show()
```

Ожидаемый результат: схема дерева с вопросами в узлах и предсказаниями в листьях.

Переобучение и параметр max_depth

Аналогия: зубрёжка каждой мелочи 💡

Если дереву не ограничивать глубину, оно будет задавать вопросы, пока не запомнит каждый обучающий пример по отдельности — вплоть до «этому конкретному пассажиру ставим выжил». На train будет 100%, а на новых данных — провал. Это переобучение (overfitting): модель выучила частности вместо закономерностей.

Противоположная беда — **недообучение (underfitting)**: дерево слишком простое (например, глубина 1) и не улавливает даже явных закономерностей. Задача — найти баланс. Главный рычаг здесь — max_depth, максимальная глубина дерева.

```
# Сравним дерево без ограничения и с max_depth=3
for depth in [3, None]: # None = без ограничения
    t = DecisionTreeClassifier(max_depth=depth, random_state=42)
    t.fit(X_train, y_train)
    print(f"depth={depth}: "
```

```
f"train={t.score(X_train, y_train):.2f}, "  
f"test={t.score(X_test, y_test):.2f}")
```

Ты увидишь: без ограничения train близок к 1,00, а test заметно ниже — классический разрыв, признак переобучения.

Минута этики

Дерево наглядно показывает, по каким признакам оно решает. А что, если модель принимает важные решения о людях — по признаку пола или возраста? Это повод для разговора: какие признаки использовать этично, а какие могут закрепить несправедливость. Модель делает то, чему её научили данные, — и ответственность на человеке.

Проверь себя

1. Что такое переобучение? Как его заметить по train и test?
2. Какой параметр ограничивает сложность дерева?
3. Чем недообучение отличается от переобучения?

Глава 8. Регрессия: предсказываем числа

До сих пор мы предсказывали категории — выжил или нет. Но огромный класс задач требует предсказать число: цену, температуру, оценку. Это регрессия.

Классификация против регрессии

Классификация отвечает на вопрос «какая категория?» (кошка/собака). Регрессия отвечает на вопрос «сколько?» (сколько будет стоить, сколько баллов). Если ответ — число на непрерывной шкале, это регрессия.

Линейная регрессия: линия через облако точек

Аналогия: линейка через точки

Нанеси на график точки: по горизонтали — часы учёбы, по вертикали — оценка за экзамен. Точки образуют «облако», уходящее вверх: больше учишься — выше оценка. Линейная регрессия проводит через это облако прямую линию так, чтобы она проходила максимально близко ко всем точкам. По этой линии можно предсказать оценку для любого числа часов.

Регрессия в коде на синтетических данных

Возьмём простые данные «часы учёбы → баллы» — они идеально показывают идею.

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Синтетические данные: часы учёбы и баллы (с небольшим шумом)
hours = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
scores = np.array([20, 35, 40, 55, 60, 72, 80, 88])

model = LinearRegression()
model.fit(hours, scores) # находим лучшую прямую

# Рисуем точки и линию модели
plt.scatter(hours, scores, label="реальные данные")
plt.plot(hours, model.predict(hours), color="red", label="линия модели")
plt.xlabel("Часы учёбы"); plt.ylabel("Баллы"); plt.legend()
plt.show()

# Предсказание: сколько баллов при 5.5 часах учёбы?
print("Прогноз:", model.predict([[5.5]]))
```

Ожидаемый результат: график с точками и красной линией + прогноз около 66 баллов.

Метрики регрессии: MAE и MSE

В классификации мы считали долю правильных ответов. В регрессии ответ — число, и «правильно/неправильно» не подходит: важно, насколько мы промахнулись. Для этого две главные метрики.

Метрика	Что это	Особенность
MAE	Средняя абсолютная ошибка — на сколько в среднем промахиваемся	В единицах задачи (баллы, рубли). Легко понять
MSE	Средний квадрат ошибки	Сильно наказывает крупные промахи

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

pred = model.predict(hours)
print("MAE:", mean_absolute_error(scores, pred))
print("MSE:", mean_squared_error(scores, pred))
```

Полезно знать

MAE = 4 означает: в среднем модель ошибается на 4 балла. Это интуитивно понятно, потому что измеряется в тех же единицах, что и цель. MSE труднее толковать напрямую, зато она сильнее реагирует на большие ошибки — полезно, когда крупные промахи особенно нежелательны.

Проверь себя

1. «Предсказать завтрашнюю температуру» — это классификация или регрессия?
2. Что означает MAE = 3 в задаче предсказания оценок?
3. Почему MSE сильнее наказывает большие ошибки, чем MAE?

Глава 9. Случайный лес: сила ансамбля

Одно дерево решений легко переобучается и капризно. А что, если взять много разных деревьев и устроить голосование? Так рождается случайный лес (Random Forest) — один из самых надёжных алгоритмов классического ML.

Аналогия: голосование класса

Спроси одного ученика — он может ошибиться. Спроси весь класс и возьми ответ большинства — коллективный ответ почти всегда точнее. Это «мудрость толпы». Случайный лес строит сотни деревьев, каждое немного по-своему, и объединяет их голоса. Ошибки отдельных деревьев взаимно гасятся.

Почему деревья должны быть разными

Если бы все деревья были одинаковыми, голосование ничего не дало бы — они ошибались бы хором. Поэтому лес специально делает деревья разнообразными: каждое учится на случайной части данных и случайном наборе признаков. Разнообразие — источник силы ансамбля.

Эксперимент-викторина

Задайте классу вопрос с числовым ответом («сколько окон в школе?»). Запишите ответы всех и посчитайте среднее. Почти наверняка среднее по классу окажется ближе к правде, чем большинство отдельных ответов. Вы наглядно показали принцип ансамбля.

Случайный лес в коде

```
from sklearn.ensemble import RandomForestClassifier

# n_estimators — сколько деревьев в лесу
forest = RandomForestClassifier(n_estimators=100, random_state=42)
forest.fit(X_train, y_train)

print("Лес, accuracy:", forest.score(X_test, y_test))
```

Важность признаков

Лес умеет сказать, какие признаки оказались важнее для предсказания, — это **feature importance**. Мощный инструмент, чтобы понять данные.

```
import pandas as pd

# Насколько каждый признак важен для леса
importances = pd.Series(forest.feature_importances_, index=X.columns)
print(importances.sort_values(ascending=False))
```

Турнир алгоритмов

Мы изучили три классификатора. Устроим честный турнир на одних и тех же train/test и сравним.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

```
models = {
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Дерево": DecisionTreeClassifier(max_depth=3, random_state=42),
    "Лес": RandomForestClassifier(n_estimators=100, random_state=42),
}
for name, m in models.items():
    m.fit(X_train, y_train)
    print(f"{name}: {m.score(X_test, y_test):.3f}")
```

Чаще всего лес выигрывает — но не всегда. Иногда простая модель не хуже, а понятнее. Это важный урок: сложнее не значит лучше.

Проверь себя

1. Почему ансамбль деревьев точнее одного дерева?
2. За что отвечает параметр `n_estimators`?
3. Что показывает `feature importance`?

Глава 10. Pipeline и кросс-валидация

Правило курса

Начиная с этой главы, все модели в книге собираются через Pipeline. Это профессиональный стандарт, который исключает целый класс ошибок — в первую очередь утечку данных. Дальше без Pipeline мы не работаем.

Зачем нужен Pipeline

Аналогия: конвейер на заводе

Pipeline — это конвейер: данные входят с одного конца, проходят через одни и те же станки (заполнить пропуски → закодировать текст → отмасштабировать → модель) и выходят готовым предсказанием. Собрав конвейер один раз, ты гарантируешь, что и на обучении, и на тесте, и в реальном приложении данные обрабатываются абсолютно одинаково.

Главное: Pipeline «настраивает» преобразования только на train (например, вычисляет среднее для заполнения пропусков) и применяет их к test — а не подсматривает в test заранее. Именно ручное масштабирование до разделения было источником утечки из главы 6. Pipeline закрывает эту дыру автоматически.

Масштабирование признаков

Вспомни проблему KNN: возраст 0–80 и цена 0–500 несопоставимы по масштабу. Масштабирование приводит все признаки к сравнимому диапазону, чтобы ни один не «перевешивал» просто из-за крупных чисел. Самый частый инструмент — StandardScaler.

Собираем первый Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsClassifier

# Конвейер из трёх станков:
pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")), # заполнить пропуски
    ("scale", StandardScaler()),                 # отмасштабировать
    ("model", KNeighborsClassifier(n_neighbors=5)), # модель
])

pipe.fit(X_train, y_train) # весь конвейер обучается разом
print("Pipeline accuracy:", pipe.score(X_test, y_test))
```

Теперь пропуски и масштаб обрабатываются честно, без утечки.

Разные типы столбцов: ColumnTransformer

В реальных данных числа и текст соседствуют: возраст (число) и пол (текст «male»/«female»). Числа масштабируют, текст — кодируют в числа. ColumnTransformer позволяет применить разные преобразования к разным столбцам внутри одного конвейера.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
```

```

num = ["Age", "Fare"]      # числовые столбцы
cat = ["Sex"]              # текстовые столбцы

prep = ColumnTransformer([
    ("num", StandardScaler(), num),
    ("cat", OneHotEncoder(), cat),  # текст → числа (0/1)
])

full = Pipeline([("prep", prep),
                  ("model", RandomForestClassifier(random_state=42))])

```

Кросс-валидация: одна проверка — ненадёжно

Аналогия: одна контрольная или четыре 💡

Оценить ученика по одной контрольной ненадёжно — вдруг попался неудачный вариант. Четыре контрольные по разным темам дают честную картину. Кросс-валидация делает то же: делит данные на несколько частей (folds), по очереди обучает на одних и проверяет на другой, а потом усредняет результат.

```

from sklearn.model_selection import cross_val_score

# cv=5 → делим данные на 5 частей, проверяем 5 раз
scores = cross_val_score(full, X, y, cv=5)
print("Оценки по фолдам:", scores.round(3))
print("Среднее:", scores.mean().round(3))

```

Пять чисел вместо одного показывают не только средний результат, но и его стабильность. Если оценки скачут (0,60 и 0,80) — модель неустойчива, и одному запуску верить нельзя.

Соревнование класса

Возьмите общий датасет (например, House Prices с Kaggle). Каждый собирает свой Pipeline и настраивает модель, а победитель определяется по средней оценке кросс-валидации — не по одному везучему split. Так соревнование становится честным.

Проверь себя ⓘ

1. Как Pipeline защищает от утечки данных?
2. Зачем масштабировать признаки перед KNN?
3. Чем оценка по кросс-валидации надёжнее одного train/test split?

Глава 11. Метрики и матрица ошибок

Мы всё время мерили качество через ассигасу — долю правильных ответов. Пора узнать, почему одной ассигасу категорически недостаточно и как понять, какие именно ошибки делает модель.

Провокация: 99% и полностью бесполезно

Аналогия: детектор редкой болезни 💡

Болезнь встречается у 1 человека из 100. Сделаем «модель», которая всем говорит «здоров». Её ассигасу — 99%! Но она не нашла ни одного больного и абсолютно бесполезна. Вот почему на несбалансированных данных ассигасу врёт: большой класс перекрывает всё.

Матрица ошибок (confusion matrix)

Чтобы увидеть, где именно модель ошибается, строят матрицу ошибок — таблицу 2×2, которая раскладывает все ответы модели на четыре типа.

	Модель: «да»	Модель: «нет»
На самом деле «да»	Верно (TP)	Пропуск (FN)
На самом деле «нет»	Ложная тревога (FP)	Верно (TN)

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

pred = full.predict(X_test) # full — наш Pipeline из главы 10
cm = confusion_matrix(y_test, pred)
ConfusionMatrixDisplay(cm).plot()
plt.show()
```

Precision, Recall и F1

Из матрицы выводятся три метрики, которые говорят гораздо больше, чем ассигасу.

Метрика	Отвечает на вопрос	Когда важна
Precision	Из тех, кого модель назвала «да», сколько реально «да»?	Когда дорога ложная тревога
Recall	Из всех реальных «да» сколько модель нашла?	Когда дорого пропустить
F1	Баланс между precision и recall	Когда важно и то, и другое

Аналогия: спам-фильтр против детектора болезни 💡

В спам-фильтре важнее precision: лучше пропустить пару спам-писем, чем отправить важное письмо в спам (ложная тревога дорога). В детекторе болезни важнее recall: лучше лишний раз перепроверить здорового, чем пропустить больного. Метрику выбирают под цену ошибки в конкретной задаче.

```
from sklearn.metrics import classification_report

# Сразу все метрики по каждому классу
print(classification_report(y_test, pred))
```

Как выбрать главную метрику

Универсального ответа нет — всё зависит от задачи и цены ошибки. Прежде чем оценивать модель, задай вопрос: что дороже — ложная тревога или пропуск? Ответ и подскажет метрику. Это, пожалуй, главный навык взрослого специалиста по ML.

Проверь себя

1. Почему accuracy 99% может означать бесполезную модель?
2. Чем precision отличается от recall?
3. Для детектора мошеннических операций какая метрика важнее и почему?

Глава 12. Мини-проект №2: «Предсказатель»

Вторая фаза завершается настоящим проектом. Здесь ты один раз проходишь весь путь, который специалисты по данным проходят каждый день: от сырых данных до обоснованного вывода — с обязательным Pipeline.

Полный цикл проекта

Твой проект должен пройти пять шагов. Это скелет любого ML-проекта, и мы будем повторять его до автоматизма.

1. Постановка задачи. Что предсказываем и зачем? Классификация или регрессия?
2. Данные. Выбрать датасет, изучить, почистить (пропуски, выбросы).
3. Baseline. Построить тупую модель как точку отсчёта.
4. Модель в Pipeline. Собрать конвейер и оценить по кросс-валидации.
5. Вывод. Сравнить с baseline, выбрать метрику, объяснить результат словами.

Шаблон проекта

Вот каркас, который остаётся заполнить своими данными. Он объединяет всё из фазы 2.

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier

# 1. Данные — подставь свой датасет
df = pd.read_csv("МОЙ_ДАТАСЕТ.csv")
X = df.drop(columns=["ЦЕЛЬ"])
y = df["ЦЕЛЬ"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 2. Baseline
base = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print("Baseline:", base.score(X_test, y_test))

# 3. Pipeline (укажи свои числовые и текстовые столбцы)
num = ["ЧИСЛО1", "ЧИСЛО2"]
cat = ["ТЕКСТ1"]
prep = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc", StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat),
])
model = Pipeline([("prep", prep),
                  ("clf", RandomForestClassifier(random_state=42))])

# 4. Честная оценка
```

```
scores = cross_val_score(model, X, y, cv=5)
print("Модель (кросс-валидация):", scores.mean().round(3))
```

Три слайда для защиты

Оформи результат как мини-презентацию: задача → данные и результат → вывод.
Обязательно сравни свою модель с baseline — это показывает, что ты понимаешь, чего стоит результат.

Как оценивается

Критерий	Баллы	Что оценивается
Постановка задачи	0-2	Чётко, что предсказываем
Данные и Pipeline	0-3	Чистка, корректный split, Pipeline
ML и сравнение с baseline	0-3	Модель обучена и сравнена с базовой
Визуализация и метрики	0-2	Графики и правильная метрика
Объяснение результата	0-3	Вывод словами, понимание ошибок

Главная мысль фазы 2 🎯

Ты умеешь строить модели KNN, деревья, регрессию и случайный лес, собирать их в Pipeline, честно проверять через train/test и кросс-валидацию, сравнивать с baseline и выбирать правильную метрику. Это полный набор навыков классического ML. Дальше — нейросети.

ФАЗА 3

Глубокое обучение

Разбираемся, как нейросети «видят» изображения и учатся на примерах.

Глава 13. Нейросети: интуиция без уравнений

Классические модели из второй фазы отлично работают на таблицах. Но как научить машину узнавать лица, понимать речь, различать картинки? Здесь на сцену выходят нейронные сети — идея, вдохновлённая тем, как устроен мозг.

Аналогия: ученик, исправляющий ошибки 💡

Нейросеть учится, как школьник, который решает задачу, сверяется с ответом, видит, где ошибся, и чуть-чуть корректирует свой подход. Потом снова, и снова, тысячи раз. С каждым разом ошибок всё меньше. Никто не даёт сети готовых правил — она сама подстраивается под данные, шаг за шагом уменьшая ошибку.

Нейрон — это взвешенная сумма

Искусственный нейрон гораздо проще биологического. Он берёт несколько чисел на вход, умножает каждое на свой **вес** (насколько этот вход важен), складывает всё вместе и пропускает результат через функцию, решающую, «сработать» нейрону или нет.

Аналогия: приёмная комиссия 💡

Нейрон — как член приёмной комиссии, который взвешивает факторы об абитуриенте: оценки $\times 0,5$, олимпиады $\times 0,3$, собеседование $\times 0,2$. Он складывает взвешенные баллы и выносит решение. Вес показывает, насколько для него важен каждый фактор. Обучение сети — это подбор правильных весов.

Слои: от простого к сложному

Один нейрон умеет мало. Но если выстроить нейроны в слои — где выход одного слоя становится входом следующего — сеть научается распознавать всё более сложные вещи. Первый слой ловит простые детали (края, точки), следующие складывают из них узоры, а последний выдаёт ответ.

Слой	Роль
Входной	Принимает данные (например, пиксели картинки)
Скрытые	Находят закономерности, от простых к сложным
Выходной	Выдаёт ответ (например, «это цифра 7»)

Как сеть учится: подкрутить веса

Сеть делает предсказание, сравнивает его с правильным ответом и вычисляет **ошибку**. Затем специальный алгоритм — **обратное распространение (backpropagation)** — определяет, какие веса виноваты в ошибке и в какую сторону их чуть подправить. Один такой проход по всем данным называется **эпохой (epoch)**. Обучение — это много эпох подряд.

Поиграй в TensorFlow Playground

Прежде чем писать код, зайти на сайт playground.tensorflow.org. Это визуальная песочница: добавляй слои и нейроны, нажимай «play» и смотри, как сеть на глазах учится разделять точки. Пять минут игры дают больше интуиции, чем страница текста.

Первая нейросеть на Keras

Keras — библиотека, которая делает создание нейросетей почти конструктором. Сеть собирается как башня из кубиков-слоёв. Инструмент `Sequential` — это стопка слоёв друг за другом, `Dense` — обычный полносвязный слой.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Стопка слоёв: 8 нейронов -> 8 нейронов -> 1 ответ
model = Sequential([
    Dense(8, activation="relu", input_shape=(4,)), # входной слой
    Dense(8, activation="relu"),                   # скрытый слой
    Dense(1, activation="sigmoid"),                 # выход: 0..1
])

# Настраиваем, как учиться
model.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary() # печатает структуру сети
```

Ожидаемый результат: таблица со слоями сети и числом параметров.

Полезно знать

`activation="relu"` — функция активации, которая позволяет сети улавливать сложные, нелинейные закономерности. Без неё сеть из многих слоёв работала бы как один простой слой. Пока достаточно запомнить: `relu` ставят в скрытых слоях почти всегда.

Проверь себя

1. Что такое вес нейрона и что происходит с весами при обучении?
2. Что такое эпоха?
3. Зачем сети нужны скрытые слои, а не один?

Глава 14. MNIST: нейросеть узнаёт цифры

Пришло время вау-момента: мы обучим сеть распознавать рукописные цифры. Это классический «первый проект» в глубоком обучении — как «Hello, world», только сеть учится видеть.

Изображение — это матрица чисел

Аналогия: раскраска по клеточкам 💡

Для нейросети картинка — не картинка, а таблица чисел. Чёрно-белое изображение цифры 28×28 пикселей — это сетка 28 на 28, где каждое число говорит, насколько клетка тёмная: 0 — белый, 255 — чёрный. Сеть видит не «семёрку», а 784 числа. И именно в этих числах она находит узор, который отличает семёрку от единицы.

Датасет MNIST

MNIST — это 70 000 изображений рукописных цифр от 0 до 9, каждое размером 28×28 пикселей. Он встроен прямо в Keras, качать вручную ничего не нужно.

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt

# Загружаем: сразу разделён на train и test
(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("Размер train:", X_train.shape)  # (60000, 28, 28)

# Посмотрим на первую цифру глазами
plt.imshow(X_train[0], cmap="gray")
plt.title(f"Это цифра: {y_train[0]}")
plt.show()
```

Ожидаемый результат: картинка рукописной цифры с подписью её значения.

Подготовка данных

Числа пикселей идут от 0 до 255. Сети удобнее работать с числами от 0 до 1 — это то самое масштабирование, что и в фазе 2. Делим на 255.

```
# Масштабируем пиксели в диапазон 0..1
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Строим и обучаем сеть

Слой Flatten разворачивает картинку 28×28 в один ряд из 784 чисел. Дальше — знакомые Dense-слои. Последний слой из 10 нейронов — по одному на каждую цифру.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense

model = Sequential([
    Flatten(input_shape=(28, 28)),  # картинка -> 784 числа
```

```

        Dense(128, activation="relu"),      # скрытый слой
        Dense(10, activation="softmax"),    # 10 цифр: вероятности
    ])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# Обучаем 5 эпох
model.fit(X_train, y_train, epochs=5)

# Проверяем на тесте, которого сеть не видела
loss, acc = model.evaluate(X_test, y_test)
print("Точность на тесте:", round(acc, 3))

```

Ожидаемый результат: с каждой эпохой точность растёт; на тесте обычно около 0,97.

Полезно знать

softmax в последнем слое превращает выходы в вероятности по всем 10 цифрам, которые в сумме дают 1. Сеть не просто говорит «это 7», а сообщает: «на 95% это 7, на 3% это 1...». Ответом становится цифра с самой высокой вероятностью.

Где сеть уверенно ошибается

Найдём картинки, на которых модель промахнулась, — это самое поучительное.

```

import numpy as np

# Предсказания на тесте
pred = model.predict(X_test)
pred_labels = np.argmax(pred, axis=1) # цифра с max вероятностью

# Находим первую ошибку и смотрим на неё
wrong = np.where(pred_labels != y_test)[0][0]
plt.imshow(X_test[wrong], cmap="gray")
plt.title(f"Сеть: {pred_labels[wrong]}, правда: {y_test[wrong]}")
plt.show()

```

Это не магия

Иногда сеть ошибается уверенно — и на кривой, небрежно написанной цифре, и на случайном шуме, где человек вообще не видит цифры. Это важный урок: сеть не «понимает» цифры, как мы. Она нашла статистические узоры в числах. Там, где узор обманчив, она уверенно ошибётся. Нейросеть — мощный инструмент, но не разум.

Проверь себя

1. Как нейросеть «видит» картинку?
2. Зачем делить значения пикселей на 255?
3. Что делает слой softmax на выходе?

Глава 15. Свёрточные сети (CNN)

Обычная сеть из прошлой главы разворачивает картинку в длинный ряд чисел и теряет важное — где что находится. Свёрточные сети (CNN) устроены умнее: они смотрят на картинку кусочками и понимают её пространственную структуру. Именно CNN стоят за распознаванием лиц и объектов.

Аналогия: скользящий фильтр-рамка

Представь маленькую рамку-лупу, которую ты двигаешь по всей картинке слева направо, сверху вниз. В каждом положении рамка проверяет: «а нет ли здесь вертикальной линии?» или «а нет ли уголка?». Это и есть свёртка (convolution): маленький фильтр скользит по изображению и отмечает, где встретился нужный узор.

Карты признаков и пулинг

Результат работы фильтра — **карта признаков**: где на картинке нашёлся этот узор. Фильтров много, и каждый ищет своё. Затем слой **пулинга (pooling)** уменьшает карту, оставляя главное, — так сеть становится компактнее и меньше цепляется за мелочи.

Иерархия признаков

Аналогия: от кирпичей к дому

Первые свёрточные слои находят простые детали: края и линии. Следующие складывают из них узоры: углы, дуги, текстуры. Более глубокие — целые части объекта: глаз, колесо, ухо.

Последние — объект целиком. Как из кирпичей собирается стена, из стен — комната, из комнат — дом.

CNN в коде на CIFAR-10

CIFAR-10 — датасет цветных картинок 32×32: самолёты, кошки, машины и ещё 7 категорий. Соберём небольшую CNN.

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

(X_train, y_train), (X_test, y_test) = cifar10.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0

model = Sequential([
    Conv2D(32, (3, 3), activation="relu", input_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),          # уменьшаем карту признаков
    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax"),
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

```
model.fit(X_train, y_train, epochs=5)
```

Слой Conv2D — это набор фильтров-рамок, MaxPooling2D — уменьшение. Сравни точность этой CNN с обычной Dense-сетью на тех же картинках: CNN обычно заметно выигрывает, потому что учитывает расположение пикселей.

Transfer learning: стоим на плечах гигантов

Аналогия: готовый эксперт 💡

Обучить мощную сеть с нуля — это месяцы вычислений и миллионы картинок. Но есть готовые сети, уже обученные на огромных наборах данных. Transfer learning — это взять такого «эксперта», который уже умеет различать формы и текстуры, и доучить его под свою задачу за пару минут. Как нанять опытного повара и показать ему пару своих рецептов, вместо того чтобы учить готовить с нуля.

Популярная готовая сеть — **MobileNetV2**: лёгкая и точная, идеальна для учебных проектов. В следующей главе мы используем именно её, чтобы за один урок собрать классификатор собственных фотографий.

Проверь себя ⓘ

1. Что делает свёрточный фильтр, скользя по картинке?
2. Зачем нужен слой пулинга?
3. Почему transfer learning экономит время и данные?

Глава 16. Проект «Классификатор моих фото»

Соберём всё из третьей фазы в настоящий продукт: обучим сеть узнавать категории на твоих собственных фотографиях и завернём её в веб-приложение, ссылку на которое можно отправить родителям. Это первый проект, который живёт в интернете.

Шаг 1. Собери свои данные

Выбери 2–3 категории, которые тебе интересно различать: например, «кошки / собаки», «кроссовки / ботинки», твои любимые игрушки. Сними или скачай по 20–30 фотографий каждой категории и разложи по папкам с названиями категорий. Данные — половина успеха.

Аналогия: личный датасет 💡

До сих пор ты работал с чужими датасетами. Теперь ты создаёшь свой — с нуля, своими руками. Это ровно то, чем занимаются инженеры ML в реальных задачах: качество данных решает больше, чем выбор модели.

Шаг 2. Аугментация данных

Аналогия: одно фото — много ракурсов 💡

30 фотографий — маловато. Аугментация (augmentation) искусственно расширяет датасет: слегка поворачивает, отражает, меняет яркость каждой картинки. Из одного фото получается несколько разных. Сеть видит объект под разными углами и учится узнавать его надёжнее — как если бы ты рассмотрел предмет со всех сторон, а не только спереди.

```
from tensorflow.keras.layers import RandomFlip, RandomRotation
from tensorflow.keras.models import Sequential

# Слой аугментации: случайный поворот и отражение
augment = Sequential([
    RandomFlip("horizontal"),
    RandomRotation(0.1),
])
```

Шаг 3. Дообучаем MobileNetV2

Берём готовую сеть MobileNetV2, «замораживаем» её знания и добавляем сверху свой маленький классификатор под наши категории. Это transfer learning из прошлой главы в действии.

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense
from tensorflow.keras.models import Model

# Готовый "эксперт", обученный на миллионах картинок
base = MobileNetV2(input_shape=(160, 160, 3),
                   include_top=False, weights="imagenet")
base.trainable = False # замораживаем — не переучиваем с нуля

# Добавляем свою "голову" под наши категории
x = GlobalAveragePooling2D()(base.output)
out = Dense(2, activation="softmax")(x) # 2 = число твоих категорий
model = Model(base.input, out)
```

```
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

Внимание

Загрузка готовых моделей (weights="imagenet") и работа Gradio требуют доступа в интернет — запускай эти ячейки именно в Google Colab, а не в среде без сети. В Colab всё работает из коробки.

Шаг 4. Веб-интерфейс через Gradio

Модель есть — но как её показать? **Gradio** создаёт веб-страницу с загрузкой картинки и ответом модели буквально из нескольких строк Python. И сразу даёт публичную ссылку.

```
import gradio as gr
import numpy as np
from PIL import Image

labels = ["категория_1", "категория_2"] # твои названия

def classify(img):
    img = Image.fromarray(img).resize((160, 160))
    arr = np.expand_dims(np.array(img) / 255.0, axis=0)
    pred = model.predict(arr)[0]
    return {labels[i]: float(pred[i]) for i in range(len(labels))}

# Собираем интерфейс: вход — картинка, выход — метки с вероятностями
demo = gr.Interface(fn=classify,
                   inputs=gr.Image(),
                   outputs=gr.Label())

# share=True даёт публичную ссылку, которой можно поделиться
demo.launch(share=True)
```

Ожидаемый результат: ссылка вида <https://xxxxx.gradio.live> — работающее приложение в браузере.

Отправь ссылку родителям

Главная мысль фазы 3

Ты прошёл путь от «нейрон — это взвешенная сумма» до собственного веб-приложения, которое узнаёт объекты на фотографиях. Ты понял, как сети видят через матрицы чисел, обучил модель на MNIST, построил CNN и применил transfer learning. И, главное, у тебя есть публичная ссылка на живой продукт. Отправь её родителям — это то, что можно показать и чем можно гордиться.

Проверь себя

1. Зачем нужна аугментация данных на маленьком датасете?
2. Что значит «заморозить» базовую модель при transfer learning?
3. Что делает параметр share=True в Gradio?

ФАЗА 4

Большие языковые модели

Понимаем, как работает то, что стоит за ChatGPT, и решаем реальные задачи.

Глава 17. Токены, эмбединги и внимание

ChatGPT, Claude и другие чат-боты кажутся волшебством. Но под капотом у них — большая языковая модель (LLM, Large Language Model), и её основную идею можно понять без единой формулы. В этой главе разберём, как текст превращается в числа и почему модель иногда уверенно врёт.

Аналогия: очень умный автодополнитель

В основе LLM лежит простая задача: предсказать следующее слово. Как автодополнение в телефоне, только несравнимо мощнее. «Небо сегодня...» → «голубое». Модель прочитала гигантские объёмы текста и научилась угадывать, что идёт дальше. Всё, что делает чат-бот, — это раз за разом предсказывает следующее слово. Из этого простого механизма и рождается связный ответ.

Шаг 1. Токены — кусочки текста

Модель не работает с буквами или словами напрямую. Сначала текст режется на **токены** — кусочки, обычно чуть меньше слова. «Машинное обучение» может стать токенами «Машин», «ное», « обуч», «ение». Каждому токену присвоен номер. Так текст превращается в последовательность чисел, с которой умеет работать нейросеть.

Полезно знать

Именно поэтому у моделей есть «лимит токенов», а не слов. И поэтому за длинные запросы к платному API берут больше — стоимость считается по числу токенов. Грубо: 1000 токенов — это примерно 750 английских слов.

Шаг 2. Эмбединги — смысл в виде чисел

Аналогия: карта смыслов

Эмбединг (embedding) — это способ записать смысл слова набором чисел, координатами на огромной «карте смыслов». На этой карте похожие по смыслу слова стоят рядом: «кот», «кошка», «котёнок» — в одном районе, «автомобиль», «машина», «грузовик» — в другом. Модель понимает не буквы, а расположение на этой карте. Близкие числа — близкий смысл.

Из-за такой геометрии смыслов возможны почти магические штуки: «король» – «мужчина» + «женщина» ≈ «королева». Модель улавливает не слова, а отношения между смыслами.

Шаг 3. Внимание — на какие слова смотреть

Аналогия: на какие слова обращать внимание

В предложении «Кошка не поймала мышь, потому что она была слишком быстрой» — кто «она»? Кошка или мышь? Человек понимает по смыслу: раз не поймала, «быстрой» была мышь. Механизм внимания (attention) позволяет модели при обработке каждого слова смотреть на другие слова предложения и решать, какие из них сейчас важны. Слово «она» «обращает внимание» на «мышь». Это и есть сердце трансформера.

Архитектура, которая построена вокруг механизма внимания, называется **трансформер (transformer)**. Это она стоит за буквой Т в «ChatGPT» (Generative Pretrained Transformer). Именно внимание позволило моделям понимать длинный связный текст, а не только соседние слова.

Посмотри на внимание своими глазами: BertViz

Внимание можно не только вообразить, но и увидеть. Инструмент BertViz рисует линии между словами: чем ярче линия, тем сильнее одно слово «смотрит» на другое. Запусти в Colab и попробуй разные предложения — видно, как местоимения тянутся к своим существительным.

```
# Запускать в Google Colab (нужен интернет для загрузки модели)
!pip install bertviz transformers -q

from transformers import AutoTokenizer, AutoModel
from bertviz import head_view

name = "bert-base-uncased"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModel.from_pretrained(name, output_attentions=True)

text = "The cat did not catch the mouse because it was fast"
inputs = tok(text, return_tensors="pt")
out = model(**inputs)

# Рисуем карту внимания между словами
tokens = tok.convert_ids_to_tokens(inputs["input_ids"][0])
head_view(out.attentions, tokens)
```

Ожидаемый результат: интерактивная схема связей между словами предложения.

Почему LLM галлюцинируют

Аналогия: уверенный отличник, который не знает ⚠

Вспомни: модель всего лишь предсказывает правдоподобное следующее слово. Она не проверяет факты — она генерирует текст, который звучит убедительно. Поэтому LLM может уверенно выдумать несуществующую книгу, дату или цитату. Это называется галлюцинацией. Модель — как отличник, который на любой вопрос отвечает гладко и уверенно, даже когда не знает ответа: главное — чтобы звучало правильно.

Из этого следует главное правило работы с LLM: никогда не доверяй фактам от модели слепо. Проверяй важные цифры, имена, ссылки. Модель отлично формулирует, переписывает, придумывает — но не является источником истины.

Проверь себя ⓘ

1. Какую базовую задачу решает языковая модель?
2. Что такое эмбединг и почему похожие слова оказываются «рядом»?
3. Почему модель может уверенно выдать ложный факт?

Глава 18. LLM на практике: промптинг и классификация текста

Теперь применим LLM к настоящей задаче. И это будет не «ещё один чат-бот»: мы научимся управлять моделью через грамотные промпты и решим реальную задачу — автоматический анализ тональности отзывов.

Промпт — это инструкция для модели

Аналогия: задание для стажёра 💡

Промпт (prompt) — это то, что ты пишешь модели. Представь, что даёшь задание толковому, но незнакомому стажёру: чем яснее и подробнее объяснишь, тем лучше результат. «Сделай нормально» — плохое задание. «Определи тональность отзыва: ответь одним словом — позитив или негатив» — хорошее. Качество ответа модели напрямую зависит от качества твоего промпта.

Три приёма хорошего промпта

1. Дай роль

«Ты — опытный редактор». Роль настраивает модель на нужный стиль и уровень ответа. Это как сказать стажёру, в качестве кого он работает.

2. Приведи примеры

Покажи модели 2–3 примера «вход → правильный ответ». Модель схватывает образец и повторяет его. Этот приём называют few-shot («несколько примеров»).

3. Попроси рассуждать по шагам

«Объясни ход мысли шаг за шагом» — для сложных задач это заметно повышает точность. Модель, «проговаривая» рассуждение, реже ошибается. Приём называют цепочкой мыслей (chain of thought).

Приём	Плохой промпт	Хороший промпт
Роль	Проверь текст	Ты редактор. Найди ошибки в тексте
Примеры	Определи тон	«Супер!» → позитив. «Ужас» → негатив. «Нормально» → ?
Формат	Расскажи про отзыв	Ответь ОДНИМ словом: позитив или негатив

Подключаемся к API

Чтобы обращаться к модели из кода, используют **API** — программный доступ к модели по ключу. Ключ выдаётся в личном кабинете провайдера (OpenAI, Anthropic и др.). Ниже — пример для Anthropic API; для OpenAI код почти такой же по смыслу.

Внимание: береги ключ ⚠️

API-ключ — как пароль от банковской карты. Никогда не публикуй его в открытом коде, не выкладывай на GitHub и не показывай в презентации. В Colab храни ключ в разделе «Секреты» (значок ключа слева), а не прямо в ячейке. За утёртый ключ платить будешь ты.

```
# Запускать в Google Colab
!pip install anthropic -q

import anthropic
from google.colab import userdata # безопасное хранилище ключей

client = anthropic.Anthropic(api_key=userdata.get("ANTHROPIC_API_KEY"))

def ask(prompt):
    msg = client.messages.create(
        model="claude-3-5-haiku-latest",
        max_tokens=50,
        messages=[{"role": "user", "content": prompt}],
    )
    return msg.content[0].text

print(ask("Ответь одним словом: столица Франции?"))
```

Ожидаемый результат: Париж.

Главный кейс: анализ тональности отзывов

Это настоящая бизнес-задача: у магазина тысячи отзывов, и надо быстро понять, каких больше — довольных или недовольных. Сделаем классификатор тональности на промптах.

```
reviews = [
    "Отличный товар, всем рекомендую!",
    "Пришло сломанное, ужасный сервис.",
    "Ну, в целом нормально, ничего особенного.",
    "Быстрая доставка, качество супер!",
]

def sentiment(review):
    prompt = (
        "Ты анализируешь отзывы. Определи тональность.\n"
        "Ответь ОДНИМ словом: позитив, негатив или нейтрально.\n\n"
        "Примеры:\n"
        "«Всё супер» -> позитив\n"
        "«Кошмар, деньги на ветер» -> негатив\n\n"
        f"Отзыв: «{review}»"
    )
    return ask(prompt).strip()

for r in reviews:
    print(f"{sentiment(r):12} | {r}")
```

Ожидаемый результат: у каждого отзыва проставлена тональность.

Обрати внимание: мы не обучали модель — она уже умеет понимать язык. Мы лишь грамотно объяснили задачу через промпт с ролью и примерами. В этом сила LLM: одна модель решает тысячи разных задач без переобучения.

Бот с характером

Роль в промпте задаёт не только точность, но и характер. Меняя **системную инструкцию**, ты получаешь бота с любой личностью — от строгого учителя до пирата. Попробуй и убедись, как сильно роль меняет тон ответов.

```
def bot(user_message, character):
    msg = client.messages.create(
        model="claude-3-5-haiku-latest",
        max_tokens=200,
        system=character, # характер бота задаётся здесь
        messages=[{"role": "user", "content": user_message}],
    )
    return msg.content[0].text

pirate = "Ты пират. Отвечай как морской волк, с 'йо-хо-хо'."
print(bot("Расскажи, что такое машинное обучение", pirate))
```

Для сильных учеников: мини-RAG по тексту

Аналогия: открытая книга на экзамене 💡

Обычная LLM отвечает по памяти и может галлюцинировать. RAG (Retrieval-Augmented Generation) даёт модели «шпаргалку»: сначала находим нужный кусок из твоего документа, затем передаём его модели вместе с вопросом. Это как экзамен с открытой книгой — модель отвечает не по памяти, а по конкретному тексту, и врёт гораздо меньше.

```
# Мини-RAG "на пальцах": вставляем нужный текст прямо в промпт
document = """
Курс длится 3 месяца и состоит из 24 занятий.
Финальный проект защищается на Demo Day перед родителями.
"""

question = "Сколько занятий в курсе?"

prompt = (
    "Ответь на вопрос ТОЛЬКО по тексту ниже. "
    "Если ответа в тексте нет — так и скажи.\n\n"
    f"Текст:\n{document}\n\n"
    f"Вопрос: {question}"
)

print(ask(prompt))
```

Это упрощённый RAG: настоящие системы автоматически ищут нужный фрагмент среди тысяч страниц по эмбедингам из главы 17. Но идея та же — заземлить ответ модели на реальный текст.

Конкурс: лучший классификатор

Устройте соревнование: у кого промпт точнее размечает набор из 20 отзывов? Побеждает не самый длинный промпт, а самый продуманный — с чёткой ролью, хорошими примерами и ясным форматом ответа.

Главная мысль фазы 4 🎯

Ты понял, как LLM превращает текст в токены и эмбединги, как работает внимание и почему модель галлюцинирует. И ты научился главному прикладному навыку 2026 года — управлять моделью через промпты и решать реальные задачи через API, критически оценивая ответы. Дальше мы научимся быстро строить модели с AutoML и выкладывать приложения в интернет.

Проверь себя ⓘ

1. Назови три приёма хорошего промпта.
2. Почему API-ключ нельзя публиковать в коде?

3. Как RAG уменьшает галлюцинации модели?

ФАЗА 5

AutoML и AI-инструменты

Ускоряем работу готовыми инструментами и выкладываем приложение в интернет.

Глава 19. AutoML: ускоряем работу

Во второй фазе ты вручную собирал Pipeline, выбирал модель, подбирал параметры. Это важный навык — но что, если нужно получить хороший результат быстро? На помощь приходит AutoML: инструменты, которые сами перебирают модели и настройки за тебя.

Аналогия: автопилот в машине

AutoML — это автопилот. Ты по-прежнему должен знать, куда едешь и как устроена машина (иначе не заметишь, если автопилот везёт не туда). Но рутину — держать полосу, подстраивать скорость — берёт на себя система. AutoML сам пробует десятки моделей и комбинаций параметров и выдаёт лучшую. Твоя задача — грамотно поставить цель и проверить результат.

Что AutoML делает за тебя

AutoML автоматизирует самую трудоёмкую часть работы: он сам пробует разные алгоритмы (лес, бустинг, KNN и другие), сам подбирает их параметры, сам сравнивает результаты по кросс-валидации и выбирает победителя. То, на что вручную уходят часы, он делает за минуты.

Этап	Вручную (фаза 2)	С AutoML
Выбор модели	Пробуешь по одной сам	Перебирает десятки автоматически
Настройка параметров	GridSearchCV вручную	Подбирает сам
Сравнение	Считаешь и сравниваешь сам	Строит рейтинг моделей

AutoML в коде

Популярные инструменты — **AutoGluon** и **H2O**. Возьмём AutoGluon: он решает табличную задачу буквально в несколько строк.

```
# Запускать в Google Colab (нужен интернет для установки)
!pip install autogluon.tabular -q

import pandas as pd
from autogluon.tabular import TabularPredictor

# Данные "Титаник" — та же задача, что решали вручную
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)
train = df.sample(frac=0.8, random_state=42)
test = df.drop(train.index)

# AutoML: одна команда — и он сам перебирает модели
predictor = TabularPredictor(label="Survived").fit(train)

# Рейтинг всех опробованных моделей
predictor.leaderboard(test)
```

Ожидаемый результат: таблица-рейтинг моделей, отсортированная по точности.

Главное сравнение: вручную против AutoML

Возьми точность своей ручной модели из фазы 2 на «Титанике» и сравни с результатом AutoML на тех же данных. Обычно AutoML немного выигрывает — но не в разы. И это важный вывод.

Внимание: AutoML — не волшебство ⚠

AutoML не заменяет понимание. Он не поставит задачу за тебя, не заметит утечку данных, не выберет правильную метрику и не объяснит, почему модель ошибается. Если данные плохие — AutoML выдаст плохой результат быстро и уверенно. Инструмент экономит время эксперту, но не делает новичка экспертом. Именно поэтому мы сначала прошли фазу 2 руками.

Hugging Face Hub: библиотека готовых моделей

Аналогия: магазин приложений для ИИ 💡

Hugging Face Hub — это как магазин приложений, только для моделей машинного обучения. Тысячи готовых моделей: перевод, распознавание речи, анализ тональности, генерация картинок. Многие можно запустить в пару строк, не обучая ничего с нуля. Прежде чем строить модель самому, загляни туда — возможно, кто-то уже сделал нужное.

```
# Запускать в Google Colab
!pip install transformers -q
from transformers import pipeline

# Берём готовую модель анализа тональности одной строкой
classifier = pipeline("sentiment-analysis")

print(classifier("I love this course!"))
print(classifier("This is terrible."))
```

Ожидаемый результат: метка POSITIVE/NEGATIVE и уверенность для каждой фразы.

Слово pipeline здесь — из библиотеки Hugging Face (не путать с sklearn Pipeline из фазы 2). Оно скачивает готовую модель и сразу применяет её к тексту.

Практика: реши Kaggle-задачу за урок

Возьми любую табличную задачу с Kaggle, запусти на ней AutoGluon и получи рейтинг моделей. Затем сравни лучшую модель AutoML со своей ручной моделью и напиши вывод: где AutoML помог, а где всё равно нужно было понимание человека.

Проверь себя ⓘ

1. Что именно автоматизирует AutoML?
2. Почему AutoML не заменяет знание основ ML?
3. Что такое Hugging Face Hub и когда туда стоит заглянуть?

Глава 20. Gradio и деплой: приложение за урок

У тебя есть обученная модель. Но модель в блокноте Colab не покажешь друзьям. В этой главе мы превратим модель в настоящее веб-приложение и выложим его в интернет так, чтобы им можно было пользоваться по ссылке из любой точки мира.

Аналогия: от кухни к ресторану

Пока твоя модель живёт в блокноте — это блюдо, которое ты готовишь только для себя на кухне. Деплой (deploy) — это открыть ресторан: любой может зайти по адресу и заказать. Gradio делает «зал и меню» (интерфейс), а Hugging Face Spaces — «здание с адресом» (постоянный хостинг в интернете).

Шаг 1. Интерфейс средствами Gradio

С Gradio мы уже встречались в главе 16. Напомним суть: ты пишешь обычную Python-функцию, а Gradio строит вокруг неё веб-страницу с полями ввода и вывода. Соберём простое приложение-калькулятор на модели регрессии «часы → баллы» из главы 8.

```
# Запускать в Google Colab
!pip install gradio -q

import gradio as gr
import numpy as np
from sklearn.linear_model import LinearRegression

# Обучаем ту же модель "часы учёбы -> баллы" из главы 8
hours = np.array([1,2,3,4,5,6,7,8]).reshape(-1,1)
scores = np.array([20,35,40,55,60,72,80,88])
model = LinearRegression().fit(hours, scores)

def predict(h):
    return round(float(model.predict([[h]])[0]), 1)

# Интерфейс: ползунок на вход, число на выход
demo = gr.Interface(
    fn=predict,
    inputs=gr.Slider(0, 12, label="Часов учёбы"),
    outputs=gr.Number(label="Прогноз баллов"),
    title="Предсказатель оценок",
)
demo.launch(share=True) # share=True – временная публичная ссылка
```

Ожидаемый результат: рабочее приложение с ползунком + ссылка *.gradio.live на 72 часа.

Полезно знать

Ссылка от share=True временная — живёт около 72 часов и работает, только пока запущен твой блокнот. Для постоянного приложения, которое работает всегда, нужен настоящий хостинг. Им и займёмся дальше.

Шаг 2. Постоянный дом — Hugging Face Spaces

Hugging Face Spaces — бесплатный хостинг для ML-приложений. Ты загружаешь туда код, а Spaces сам запускает его на своём сервере и даёт постоянный адрес. Приложение работает круглосуточно, даже когда твой компьютер выключен.

Как выложить приложение — по шагам

1. Зарегистрируйся на huggingface.co (бесплатно).
2. Нажми «New Space», выбери SDK «Gradio» и придумай название.
3. Положи код приложения в файл `app.py`.
4. Перечисли нужные библиотеки в файле `requirements.txt` (например, `gradio`, `scikit-learn`).
5. Загрузи оба файла в Space — он сам всё соберёт и запустит.

Что положить в `app.py`

Файл `app.py` — это тот же код Gradio, но без `share=True`: на Spaces публичный адрес даётся автоматически. Вот минимальный пример.

```
# app.py — файл для Hugging Face Spaces
import gradio as gr
import numpy as np
from sklearn.linear_model import LinearRegression

hours = np.array([1,2,3,4,5,6,7,8]).reshape(-1,1)
scores = np.array([20,35,40,55,60,72,80,88])
model = LinearRegression().fit(hours, scores)

def predict(h):
    return round(float(model.predict([[h]])[0]), 1)

demo = gr.Interface(fn=predict,
                    inputs=gr.Slider(0, 12, label="Часов учёбы"),
                    outputs=gr.Number(label="Прогноз баллов"))
demo.launch() # на Spaces адрес выдаётся сам, share не нужен

# requirements.txt — список библиотек
gradio
scikit-learn
numpy
```

Два файла — и приложение живёт по постоянному адресу вида huggingface.co/spaces/твое-имя/название.

Внимание ⚠

Регистрация, загрузка файлов и работа Spaces требуют интернета и выполняются на сайте huggingface.co, а не в среде без сети. Код `app.py` можно написать и проверить в Colab, а затем перенести на Spaces.

Отправь ссылку родителям — прямо с урока

Главная мысль фазы 5 🎯

Ты научился ускорять работу с AutoML, брать готовые модели с Hugging Face Hub и — самое главное — выкладывать своё приложение в интернет по постоянному адресу. Теперь у тебя есть не просто модель в блокноте, а живой продукт, ссылку на который можно отправить кому угодно прямо сейчас. Это и есть то, что отличает законченный проект от учебного упражнения. Впереди — финальный проект, где ты соберёшь всё вместе.

Проверь себя ⓘ

1. В чём разница между ссылкой `share=True` и хостингом на Spaces?
2. Какие два файла нужны, чтобы выложить приложение на Hugging Face Spaces?
3. Зачем в `app.py` убирают `share=True`?

